ssv-prom-exporter — User guide

Install, configure and run the DataCore SANsymphony Prometheus exporter

Luc Blanc

2026-05-18

1. What you are looking at

`ssv-prom-exporter` is a single-binary Prometheus exporter for DataCore SANsymphony (SSV). It scrapes the SSV REST API and exposes inventory, health and performance signals on `/metrics` in the standard Prometheus exposition format.

Three deployment options are supported, all from the same Go source tree:

- **Windows** — native Windows service (no NSSM, no wrapper script); install via MSI.
- **Linux** — static binary + hardened systemd unit; install via tarball.
- **Docker** — multi-arch OCI image on GHCR, runs as nonroot uid 65532 with a `wget` HEALTHCHECK on `/metrics`.

The exporter does **not** need to run on the SSV host. Any host on the network with TCP/443 reachability to a SANsymphony management server is enough.

This guide walks an operator through day-1 install and day-2 operations. For architecture and design intent, read the deck shipped next to this document; for the full developer reference, read the project README and DECISIONS.md.

2. Requirements

- DataCore SANsymphony **PSP 20+**.
- A target host (Windows, Linux, or any container host) with network reachability to a SANsymphony mgmt server on TCP/443.
- A SANsymphony account that can list `/serverGroups`, `/servers`, `/pools`, `/virtualDisks`, `/hosts`, `/ports`, `/physicalDisks`, `/alerts`, `/monitors`, and read `/performance/{id}`. A Windows account granted SSV administrative access works.

- An ACL'ed location to drop the YAML configuration file. The recommended path is platform-specific:
    - Windows — `C:\ProgramData\ssv-prom-exporter\config.yaml`
    - Linux — `/etc/ssv-prom-exporter/config.yaml`
    - Docker — a bind-mounted host file or `-e SSV_URL=...` env vars

3. Install on Windows (MSI workflow)

The MSI installs the binary and example config to `C:\Program Files\ssv-prom-exporter\`. It deliberately does **not** register the service — registration is a separate step so credentials never land in MSI properties or in the SCM `ImagePath`.

From an **elevated** prompt (replace `X.Y.Z` with the version you downloaded from the GitHub Releases page):

```
:: 1. Run the MSI (silent).
msiexec /i ssv-prom-exporter-X.Y.Z-x64.msi /qn

:: 2. Drop the YAML config and tighten ACLs.
copy "C:\Program Files\ssv-prom-exporter\config.example.yaml" ^
     "C:\ProgramData\ssv-prom-exporter\config.yaml"
notepad "C:\ProgramData\ssv-prom-exporter\config.yaml"
icacls "C:\ProgramData\ssv-prom-exporter\config.yaml" /inheritance:r ^
       /grant:r SYSTEM:F Administrators:F

:: 3. Register the service. Only -config lands in the SCM ImagePath.
"C:\Program Files\ssv-prom-exporter\ssv-prom-exporter.exe" ^
  -install -config "C:\ProgramData\ssv-prom-exporter\config.yaml"

:: 4. Start it.
sc start ssv-prom-exporter
```

The service is created with start type `Automatic`, running as `LocalSystem`. The Event Log source is registered under the service name; logs land in **Windows Logs → Application** filtered on that source.

4. Install on Linux (systemd workflow)

The Linux tarball ships a static binary, a hardened systemd unit, and a reference `install-linux.sh` that lays everything out. As **root**:

```
tar xzf ssv-prom-exporter-vX.Y.Z-linux-amd64.tar.gz
cd     ssv-prom-exporter-vX.Y.Z-linux-amd64
./install-linux.sh
$EDITOR /etc/ssv-prom-exporter/config.yaml   # set url / user / pass
systemctl enable --now ssv-prom-exporter
```

The installer places:

- `/usr/local/bin/ssv-prom-exporter` — the static binary.
- `/etc/systemd/system/ssv-prom-exporter.service` — the systemd unit.
- `/etc/ssv-prom-exporter/config.yaml` — created from `config.example.yaml` on the first install, **never overwritten on upgrade**.

The unit runs as a `DynamicUser` (no separate `useradd` step) with `ProtectSystem=strict`, `NoNewPrivileges`, a `SystemCallFilter`, restricted address families (AF_INET / AF_INET6 / AF_UNIX only) and memory / task limits. Output goes to `journald`:

```
journalctl --unit ssv-prom-exporter -f
systemctl status ssv-prom-exporter
```

To upgrade: extract the new tarball, re-run `./install-linux.sh`. The config file is preserved; the unit is re-installed and the service restarted if it was already enabled.

5. Run with Docker

Pre-built multi-arch images (`linux/amd64` + `linux/arm64`) are published on GHCR on every release:

```
docker run --rm -p 9876:9876 \
    -e SSV_URL=https://10.0.0.1 \
    -e SSV_USER=administrator \
    -e SSV_PASS='ChangeMe!' \
    ghcr.io/lblanc/ssv-prom-exporter:latest
```

Image properties:

- Built on `alpine:3` (~34 MB final).
- Runs as nonroot uid 65532.
- Embeds `tini` so `docker stop` (SIGTERM) is forwarded cleanly to the Go runtime — graceful shutdown.
- Ships a `HEALTHCHECK` against `http://127.0.0.1:9876/metrics` (30 s interval, 30 s start grace).

Tags published: `vX.Y.Z`, `X.Y`, `latest`.

To mount a YAML config instead of passing creds through env vars:

```
docker run --rm -p 9876:9876 \
    -v /etc/ssv/config.yaml:/etc/ssv-prom-exporter/config.yaml:ro \
    ghcr.io/lblanc/ssv-prom-exporter:latest \
    -config /etc/ssv-prom-exporter/config.yaml
```

For a complete demo (exporter + Prometheus + Grafana with the five SSV dashboards), use the full-stack compose described in §9.

6. Configure

`config.yaml` is the single source of truth. See `config.example.yaml` shipped with each release for the full schema with inline comments. The most common knobs:

```
url:       https://10.0.0.1           # SSV REST base URL
user:      administrator
pass:      ChangeMe!
listen:    ":9876"                    # HTTP listen for /metrics
insecure:  true                       # SSV ships self-signed certs

# Optional — backup management nodes seeded before the first scrape.
# After that, the exporter auto-discovers them from /servers.
bases:         "10.0.0.2,10.0.0.3"
backup_cidrs:  "10.0.0.0/24"           # default = primary's /24

# Optional — failure handling.
retries:       2
retry_delay:   "200ms"

# Optional — performance collector concurrency.
perf_workers:  8
```

Precedence:

```
explicit flag  >  env var (flag default)  >  YAML  >  built-in default
```

Unknown YAML keys are rejected at load time so a typo doesn't silently leave a setting at its default.

7. Smoke-test

From the target host, with the exporter running:

```
curl http://127.0.0.1:9876/metrics | grep ssv_up
```

You should see three lines, one per collector tier:

```
ssv_up{collector="inventory"}   1
ssv_up{collector="health"}      1
ssv_up{collector="performance"} 1
```

A zero on any tier means that collector's last scrape failed. Look at `ssv_scrape_duration_seconds{collector="..."}` and the platform's log channel (Windows Event Log / `journalctl` / `docker logs`) for the reason.

One-shot REST probe without starting the HTTP server:

```
ssv-prom-exporter -ping -url https://10.0.0.1 \
                  -user administrator -pass S3cret!
```

`-ping` prints the raw `/serverGroups` JSON and exits — useful during network / credentials triage.

8. Wire to Prometheus

Add the exporter as a scrape target. The recommended label is `group`, matching the SAN group name — that label is what the bundled Grafana dashboards filter on.

```
scrape_configs:
  - job_name: ssv
    static_configs:
      - targets: ["sansymphony-host-1:9876"]
        labels:
          group: "prod"
      - targets: ["sansymphony-host-2:9876"]
        labels:
          group: "lab"
```

A 15 s scrape interval is the sweet spot:

- Faster than SSV's 30 s REST cache buys nothing (`RequestExpirationTime` on the mgmt server's `Web.config`).
- Slower than 30 s makes counter `rate()` ranges noisy.

9. Run the full Prometheus + Grafana + exporter stack

A docker-compose stack ships under `deploy/` in the repo. Two deployment modes are supported through compose profiles.

External exporters (default)

Prometheus + Grafana run from compose; the exporter runs elsewhere (typically on each SAN host).

```
cd deploy
cp .env.example .env
$EDITOR .env          # set EXPORTER_TARGETS=name=host:port,...
docker compose up -d
```

`EXPORTER_TARGETS=lab=10.0.0.10:9876,prod=10.1.0.10:9876` declares two targets; each name becomes the `group` Prometheus label.

Full stack with the exporter (`--profile full`)

The exporter ALSO runs as a service in the compose stack. Useful for demos and for sites that prefer to run everything containerized. Single SSV group, single `.env`, single command:

```
cd deploy
cp .env.example .env
# Set SSV_URL / SSV_USER / SSV_PASS / SSV_GROUP in .env.
# Leave EXPORTER_TARGETS commented out.
docker compose --profile full up -d --build
```

Prometheus auto-discovers the in-stack exporter (scraping `exporter:9876` on the compose network) and stamps the `group` label from `SSV_GROUP` (default: `full`). The exporter's `/metrics` is not published on the host by default — uncomment the `ports:` block in `deploy/docker-compose.yml` if you want to curl it locally.

The stack's Prometheus is pull-only by default. To accept inbound backfill from `prom-clip` (see §17), set `PROM_REMOTE_WRITE=1` in `.env` before bringing the stack up. That turns on both `--web.enable-remote-write-receiver` and `storage.tsdb.out_of_order_time_window` (default 7d, override with `PROM_OOO_WINDOW`). Without the OOO window Prometheus would silently drop any imported sample older than the current head block while still returning HTTP 200 on the write.

Endpoints

- Grafana — http://localhost:3000 (anonymous Viewer enabled; `admin` / `GF_ADMIN_PASSWORD` to edit).
- Prometheus — http://localhost:9090.

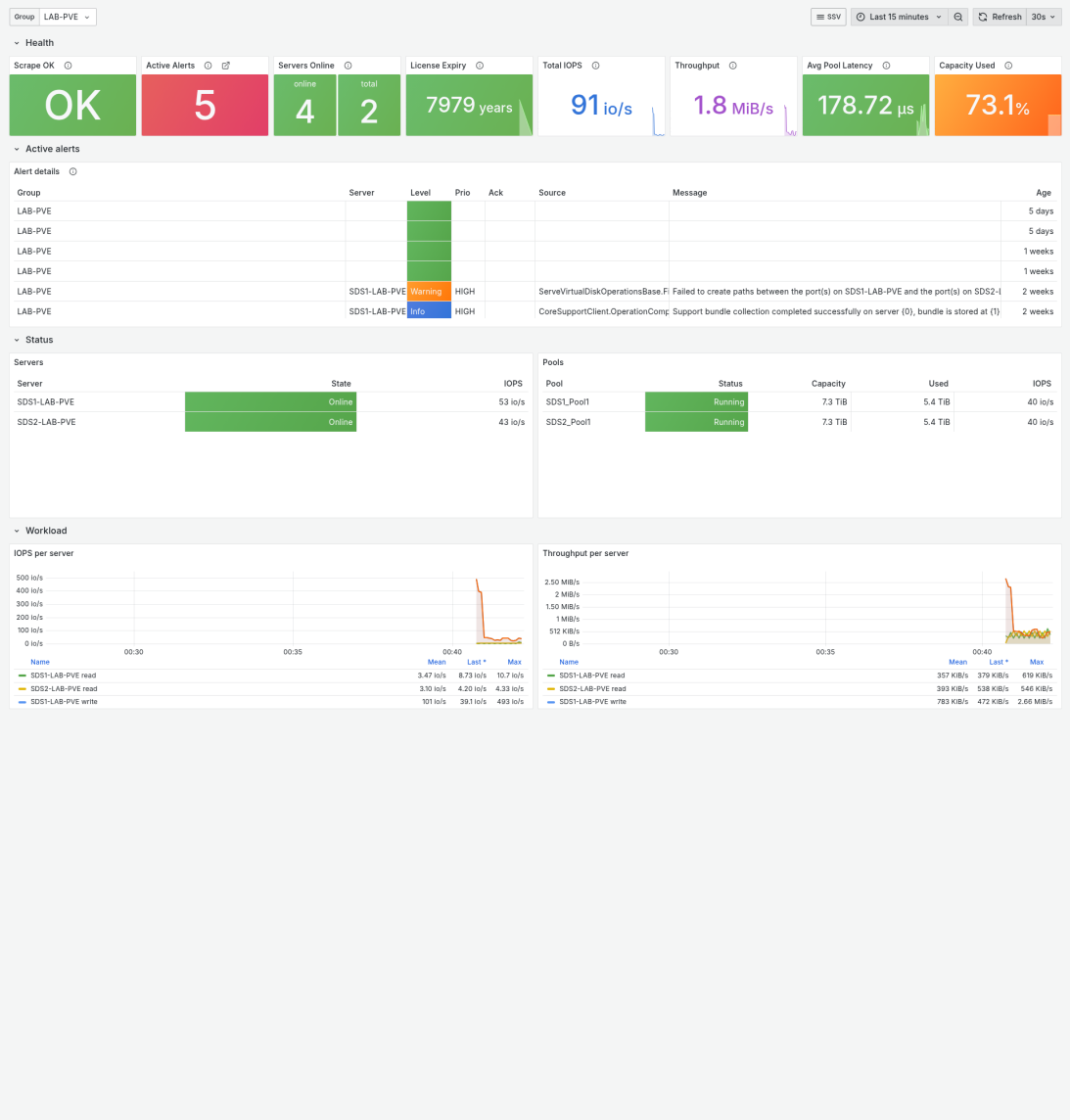
Stop the stack with `docker compose down`; add `-v` to also wipe the TSDB and Grafana volumes.

10. Grafana dashboards

Five dashboards ship pre-provisioned in the **SSV** folder, all cross-linked through an "SSV" dropdown that preserves the time range and selected filters when navigating between them. Live screenshots follow.

10.1 Overview

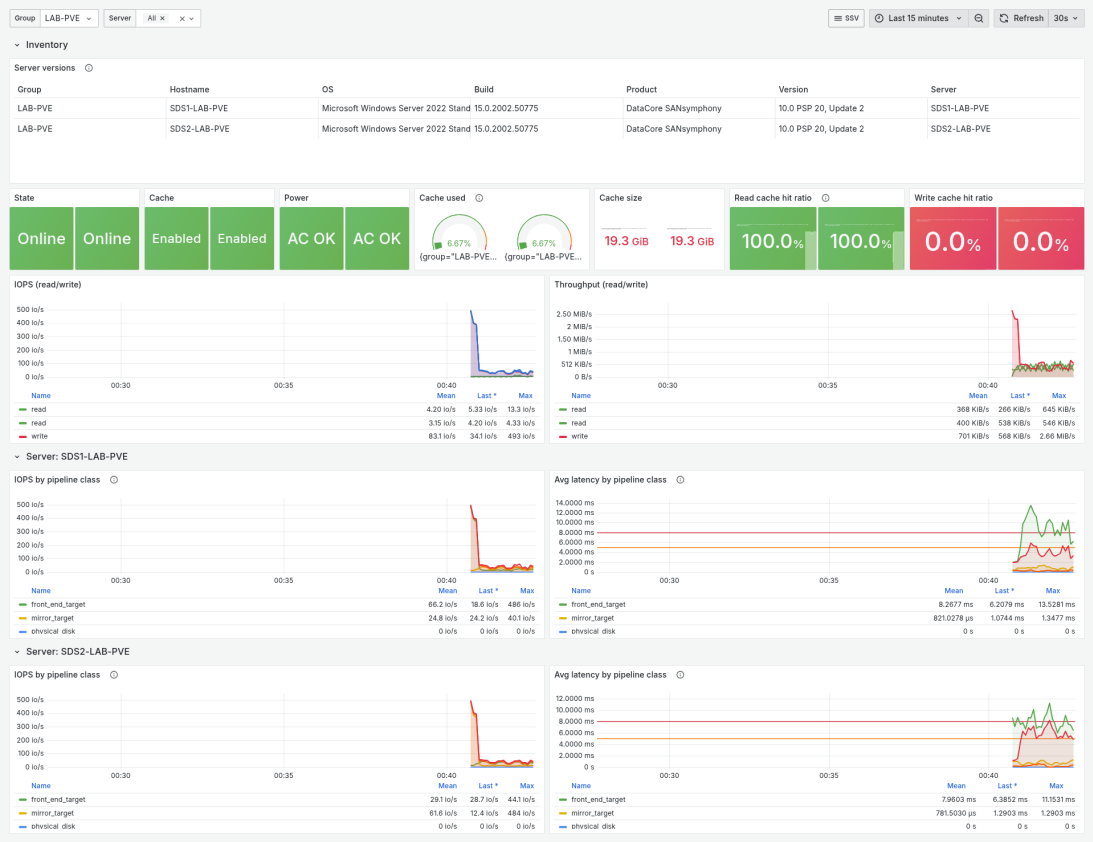
Global health: scrape, alert details (level / server / age), server states, capacity rollups, total IOPS & latency, top-N noisy vdisks, active monitors.



SSV Overview dashboard — health, alerts, capacity, IOPS

10.2 Servers

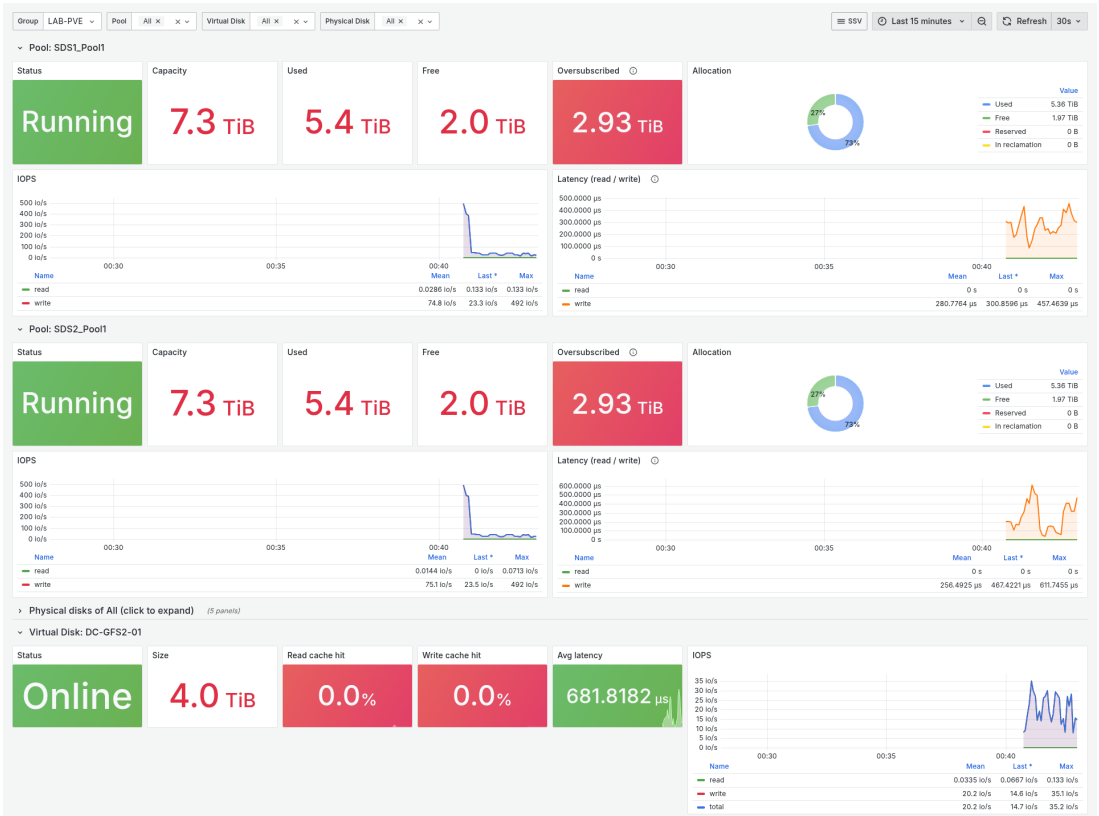
Per-server (repeated): state, cache used / size / hit ratios, IOPS & throughput, IOPS & latency by IO pipeline class (front-end target / mirror target / back-end / pool / target).



SSV Servers dashboard — per-server state, cache, IOPS by pipeline class

10.3 Storage

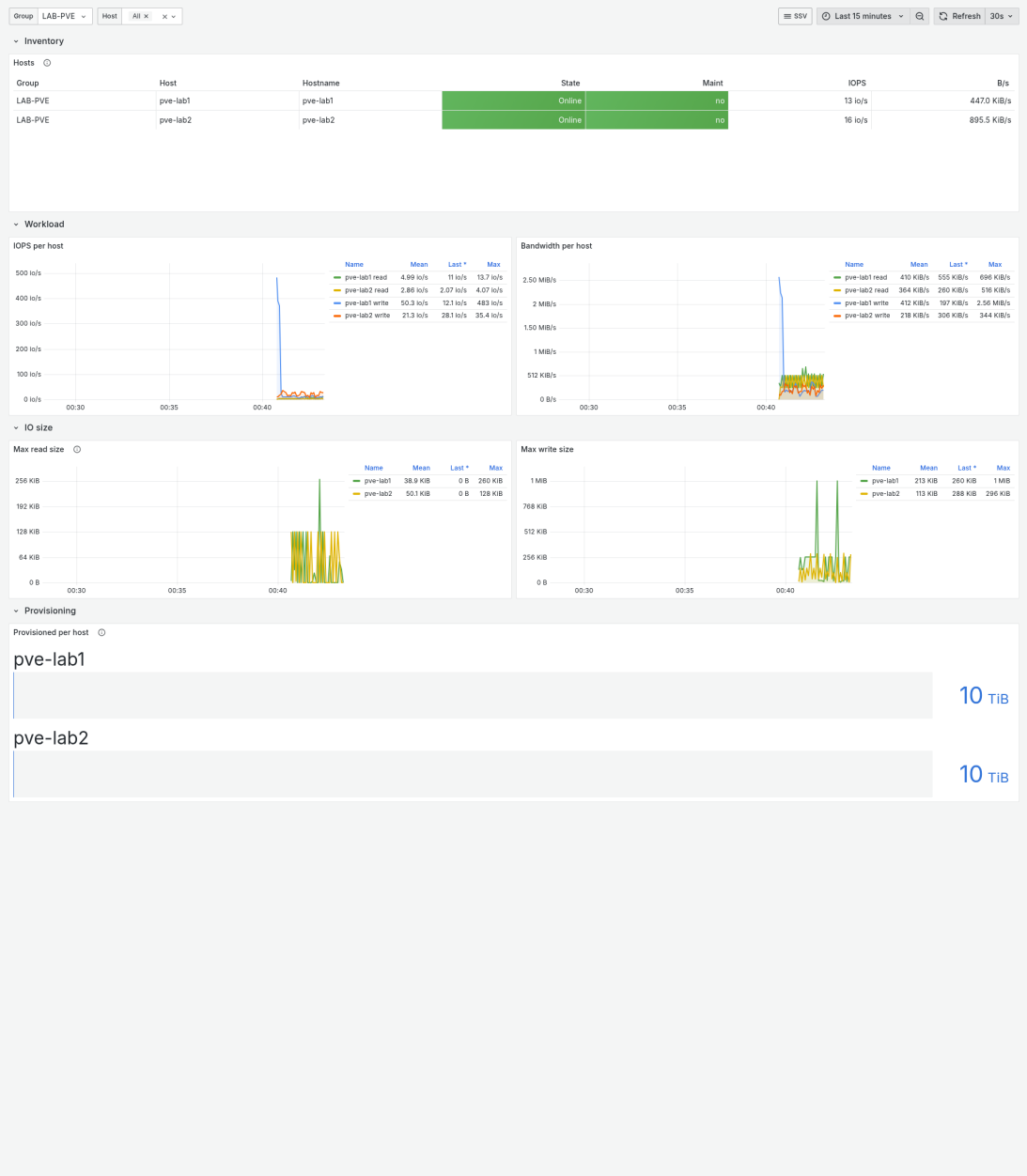
Per-pool (status, capacity pie, IOPS, latency) with a collapsible Physical disks subsection (table + per-disk IOPS / throughput / latency / queue), per-vdisk (status, cache hit ratio, IOPS, throughput, latency). Filters: Group, Pool, Virtual Disk, Physical Disk.


Group LAB-PVE
Pool All
Virtual Disk All
Physical Disk All
SSV
Last 15 minutes
Refresh 30s
Pool: SDS1_Pool1
Status
Running
Capacity
7.3 TiB
Used
5.4 TiB
Free
2.0 TiB
Oversubscribed
2.93 TiB
Allocation
Value
Used 5.36 TiB
Free 1.97 TiB
Reserved 0 B
In reclamation 0 B
IOPS
Latency (read / write)
Name Mean Last * Max
read 0.0286 io/s 0.133 io/s 0.133 io/s
write 74.8 io/s 23.3 io/s 492 io/s
read 0 s 0 s 0 s
write 280.7764 µs 300.8596 µs 457.4639 µs
Pool: SDS2_Pool1
Status
Running
Capacity
7.3 TiB
Used
5.4 TiB
Free
2.0 TiB
Oversubscribed
2.93 TiB
Allocation
Value
Used 5.36 TiB
Free 1.97 TiB
Reserved 0 B
In reclamation 0 B
IOPS
Latency (read / write)
Name Mean Last * Max
read 0.0144 io/s 0 io/s 0.0713 io/s
write 75.1 io/s 23.5 io/s 492 io/s
read 0 s 0 s 0 s
write 256.4925 µs 467.4221 µs 611.7455 µs
Physical disks of All (click to expand) (5 panels)
Virtual Disk: DC-GFS2-01
Status
Online
Size
4.0 TiB
Read cache hit
0.0%
Write cache hit
0.0%
Avg latency
681.8182 µs
IOPS
Name Mean Last * Max
read 0.0335 io/s 0.0667 io/s 0.133 io/s
write 20.2 io/s 14.6 io/s 35.1 io/s
total 20.2 io/s 14.7 io/s 35.2 io/s

SSV Storage dashboard — per-pool status, physical disks, per-vdisk

10.4 Hosts

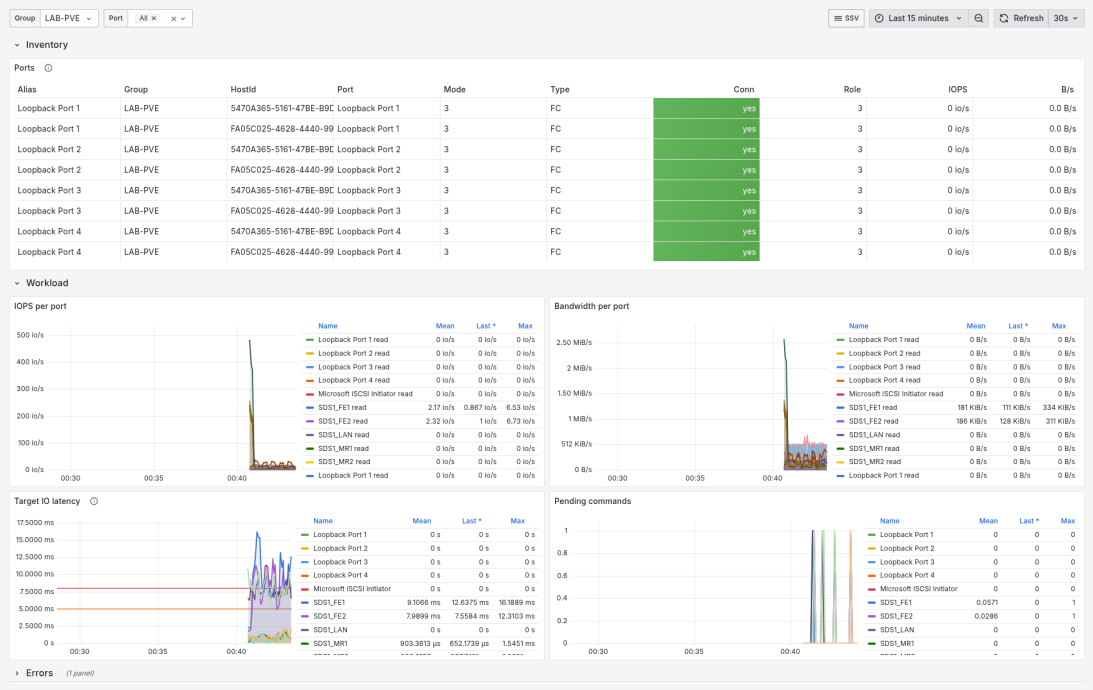
SAN-client inventory + per-host IOPS & bandwidth, peak IO size, provisioned capacity, plus a Connections (ports) subsection showing the host's ports with their IOPS & bandwidth.



SSV Hosts dashboard — SAN-client inventory, IOPS, peak IO size, provisioning

10.5 Ports

Per-port (table + IOPS + bandwidth + target IO latency + pending commands) with a collapsible Errors row plotting all link-layer counters together.



SSV Ports dashboard — per-port IOPS, target latency, pending commands

11. Metrics cheatsheet

Non-exhaustive. Open `/metrics` on a running instance for the live list.

Scrape framing

- `ssv_up{collector}` — 1 if the last tier scrape succeeded.
- `ssv_scrape_duration_seconds{collector}` — last scrape duration.

Inventory (counters & gauges describing the topology)

- `ssv_server_*` (state, cache, memory, `info{host_name, product_version, ...}`)
- `ssv_server_group_*` (state, used/max bytes, `license_expires_seconds`)
- `ssv_pool_*` (status, type, `chunk_size_bytes`)
- `ssv_virtual_disk_*` (status, size_bytes, type, offline)
- `ssv_host_*` (state, connection_state, `info{host_name, description, version}`)
- `ssv_port_*` (connected, role_capability, `info{host, port_name, alias, ...}`)
- `ssv_physical_disk_*` (status, size_bytes, free_bytes, `info{pool, tier, ...}`)

Health

- `ssv_monitor_state{monitor_id, template, target_id, caption}`
- `ssv_alerts_total` — count
- `ssv_alert_info{alert_id, machine, level, caller, message, ...}` — gauge=1 per alert
- `ssv_alert_age_seconds{alert_id}`

Performance — bytes & ops (counters)

Same family per object: `read_bytes_total`, `write_bytes_total`, `read_ops_total`, `write_ops_total`, plus object-specific extras (server cache, pool capacity, port pending/errors, physical disk queue/pending, host provisioned/peak IO size).

Performance — latency (counters / max gauges, **in seconds**)

The exporter scales SSV's millisecond timers to seconds (Prometheus convention).

- `ssv_server_class_io_{operations_total,time_seconds_total,max_time_seconds}{class}` — class ∈ {front_end_target, mirror_target, physical_disk, pool, target}
- `ssv_pool_{read,write,io}_time_seconds_total`, `ssv_pool_{read,write,io}_max_time_seconds`
- `ssv_virtual_disk_io_time_seconds_total`, `ssv_virtual_disk_io_max_time_seconds`
- `ssv_port_target_io_time_seconds_total`, `ssv_port_target_io_max_time_seconds`
- `ssv_physical_disk_{read,write,io}_time_seconds_total`, `ssv_physical_disk_{read,write,io}_max_time_seconds`

Average IO latency in PromQL:

```
rate(ssv_server_class_io_time_seconds_total[$__rate_interval])
  /
rate(ssv_server_class_io_operations_total[$__rate_interval])
```

12. Troubleshooting

Symptom	Likely cause / fix
`ssv_up{collector="inventory"} 0` at start	Wrong URL, wrong creds, or `ServerHost` header not matching the IP. Run `-ping` to see the JSON error.
All metrics are stale by N seconds	Normal — SSV's REST cache is 30 s. Don't scrape faster than that.
`HTTP 400 ErrorCode 9`	`ServerHost` header missing. Should never happen with this exporter — file an issue.
`HTTP 400` after switching `-host` to a name	Hostnames are rejected; `ServerHost` must be the **IP** the call is hitting.
TLS handshake fails	`insecure: true` is the default. Flip to `false` only when you have a CA pinned (planned flag).
`ssv_up` flips between 0 and 1 every scrape	Primary mgmt node is flapping. Check that `/servers[].IpAddresses` populates the backup list and that `backup_cidrs` covers them.
Counter `rate()` returns 0	The underlying SSV counter is in `NullCounterMap` for that object — not exposed by design.
Windows service won't start	Event Viewer → Windows Logs → Application, filter on `ssv-prom-exporter`. Most common cause: bad config path passed to `-install`.
Linux service won't start	`journalctl --unit ssv-prom-exporter -n 50`. Typical causes: config file missing, wrong YAML key, blocked by `SystemCallFilter` if you've added a custom binary.
Docker container restarts repeatedly (`Restarting`)	`docker logs <container>`. Look for the same scrape errors as above. Healthcheck unhealthy = `/metrics` not reachable: confirm `-listen` matches the EXPOSEd port.
Docker compose `--profile full` errors on `SSV_URL`	The exporter service uses `${SSV_URL:?...}` so `docker compose config` and `up` fail loud if you forgot to set it in `.env`. Set it and retry.

13. Day-2 operations

13.1 Upgrade

Windows. Install the new MSI on top of the old one; `C:\ProgramData\ssv-prom-exporter\config.yaml` is preserved. Then restart the service:

```
sc stop  ssv-prom-exporter
msiexec /i ssv-prom-exporter-X.Y.Z-x64.msi /qn
sc start ssv-prom-exporter
```

Linux. Extract the new tarball, re-run `./install-linux.sh`. The service is restarted automatically if it was already enabled.

Docker. `docker pull ghcr.io/lblanc/ssv-prom-exporter:vX.Y.Z`, then re-run the container. With `docker compose --profile full`: `docker compose pull && docker compose --profile full up -d`.

13.2 Rotate credentials

Edit `config.yaml`, then restart the service:

- Windows — `sc stop ssv-prom-exporter` + `sc start ssv-prom-exporter`
- Linux — `systemctl restart ssv-prom-exporter`
- Docker — `docker restart <container>` (or `docker compose restart exporter` in full-stack mode)

13.3 Uninstall

Windows.

```
sc stop ssv-prom-exporter
"C:\Program Files\ssv-prom-exporter\ssv-prom-exporter.exe" -uninstall
rmdir /s /q "C:\ProgramData\ssv-prom-exporter"
msiexec /x ssv-prom-exporter-X.Y.Z-x64.msi /qn
```

Linux.

```
systemctl disable --now ssv-prom-exporter
rm /etc/systemd/system/ssv-prom-exporter.service
rm /usr/local/bin/ssv-prom-exporter
rm -rf /etc/ssv-prom-exporter          # only if you want to drop config too
systemctl daemon-reload
```

Docker. `docker rm -f <container>` (and `docker compose --profile full down` in full-stack mode).

14. High availability behaviour

The exporter survives a single mgmt-node outage by failing over to another node of the same SAN group.

- After every successful inventory scrape, all IPs from `/servers[].IpAddresses` are added to the backup list, filtered by `backup_cidrs` (default = the primary's `/24`).

- On a transient failure (network error, timeout, HTTP 5xx) the next endpoint is tried. HTTP 4xx is a config bug, not an outage — it doesn't trigger failover.
- The last-known-good endpoint is sticky for 5 min, so during an outage only the first call pays the dial-timeout cost. After 5 min the next call retries the primary, detecting recovery.
- If every endpoint fails transiently in one pass, the call is retried with exponential backoff (`retries`, `retry_delay`).

15. Multi-group SSV deployments

SSV mgmt servers federate state across peer groups: a single `/serverGroups` REST call lists the local group plus every peer it has been linked to, and `/servers` blends local nodes with remote ones. The remote entries carry compound IDs of the form `<remote-group-uuid>:<server-uuid>` and have most descriptive fields empty; `/performance/{id}` is local-only.

The exporter therefore restricts per-server inventory and performance fan-out to the **local** group, identified by `OurGroup=true` in `/serverGroups`. Concrete behaviour:

- `ssv_server_group_*` still expose every group visible from the local mgmt server, local or federated peer. You can keep alerting on a peer group going unreachable.
- `ssv_server_*` (state, info, cache, storage…), `ssv_server_class_*` and the failover IP pool are scoped to the local nodes only.
- Grafana's "Server" dropdown is fed from `label_values(ssv_server_state{group=~"$group"}, server)`, so federated-peer nodes no longer show as empty rows on the Servers dashboard.

Practical consequence: **run one exporter per SSV group**. The `EXPORTER_TARGETS` Prometheus generator already accepts that shape; each entry becomes a separate `job_name` with its own `group=<name>` label, which the dashboards filter on end-to-end:

```
EXPORTER_TARGETS=HCI104=10.12.104.121:9876,HCI130=10.12.130.121:9876
```

If the API surprisingly returns no group flagged `OurGroup=true` (should never happen on a single-group install), the exporter falls back to keeping all servers, so a misbehaving API can't silently empty your inventory.

16. Security notes

- **Windows.** The SCM stores a service's command line in `ImagePath`, readable by any local admin via `sc qc <name>`. Anything passed on the install-time command line — including `-pass` — therefore leaks to local admins. Always use the YAML config workflow so only `-config <path>` lands in `ImagePath`, and tighten the ACL on `config.yaml` to `SYSTEM:F Administrators:F` (no inheritance).
- **Linux.** Keep `/etc/ssv-prom-exporter/config.yaml` at mode `0640` owned by `root:root`; the `DynamicUser` reads it via `ConfigurationDirectory=ssv-prom-exporter` once `systemd` opens it for the unit. Never put secrets in `/etc/default/`.

- **Docker.** Prefer mounted YAML over `-e SSV_PASS=...` for long-running containers — env vars show up in `docker inspect` output and in any orchestrator's pod spec. The image already runs as nonroot uid 65532.
- `insecure: true` is the default because SSV ships self-signed certs. Once your site maintains an internal PKI, plan to flip it off — the custom CA pool flag is on the roadmap.

17. Companion tool: prom-clip

A second binary, `prom-clip`, ships in this repo to **clip a time window from one Prometheus and replay it into another**. The wire format is gzipped OpenMetrics (`.txt.gz`); the replay path uses Prometheus's remote-write protocol.

Typical uses:

- snapshot a customer site that runs the exporter, mail or share the `.txt.gz`, replay it into a local lab for triage;
- ship a demo dataset alongside the dashboards;
- carry a "before / after" comparison between sites that aren't network-connected.

17.1 Two modes from one binary

Web UI (default). Run `prom-clip` with no arguments. The UI listens on `http://127.0.0.1:8088` (loopback only — no Windows Firewall prompt) and opens in the default browser. The mode is **ephemeral by default**: state lives in RAM, exports stream directly to the browser's native Save-As dialog and are removed server-side as soon as the download completes. Nothing accumulates on the host that ran the tool.

Pass `-state-dir <path>` to opt into persistent mode (last connection memorised across sessions, exports accumulated under `<state-dir>/exports`, rotation via `-keep-exports N`). On Windows that path follows the OS convention when you let prom-clip pick a default (`%LOCALAPPDATA%\prom-clip`); on Linux/macOS it follows XDG (`~/.local/state/prom-clip`).

One-shot CLI. No server, no port, no state directory. Idiomatic for scripts and CI:

```
prom-clip export -src http://prom-source:9090 \
                 -from -1h -to now -step 30s \
                 -metric '^ssv_.*' \
                 -out snapshot.txt.gz

prom-clip import -dst http://prom-target:9090 \
                 -in snapshot.txt.gz
```

`-from` / `-to` accept RFC3339 (`2026-05-21T09:00:00Z`), a Go duration interpreted as `now+d` (so `-1h` is "one hour ago"), or the literal `now`. The `-step` defaults to 15 s.

17.2 Receiver-side requirements

The target Prometheus must run with both:

- `--web.enable-remote-write-receiver` — otherwise `/api/v1/write` returns HTTP 404;
- `storage.tsdb.out_of_order_time_window` set wide enough to cover the imported window — otherwise Prometheus returns HTTP 200 on the write and **silently** drops every sample older than ~2 h (the current head block).

Both are off by default in stock Prometheus, on purpose: pull is the standard mode. The bundled `deploy/` stack exposes them behind a single env variable: set `PROM_REMOTE_WRITE=1` in `deploy/.env` (optionally with `PROM_OOO_WINDOW=7d`) before bringing the stack up.

17.3 Optional S3 push

When the export finishes, the web UI can also push the `.txt.gz` to an S3-compatible bucket configured under **Settings → S3 target** (`prom put`-style: private 24 h presigned URL, or a direct URL for a public bucket). Useful to mail a customer a single link rather than an attachment.

18. Where to go next

- Read the deck shipped next to this guide for the design intent and the AI-assisted build process.
- Open `/metrics` on a live instance to see the actual surface.
- File issues / PRs at https://github.com/lblanc/ssv-prom-exporter.